PPC-checkpoint-2

Curvature Optimisation

Once the spline is built: ⟨ replace the boundary conditions with $f^{n+1}(p) = 0$

our goal is to optimize the spline such that the total curvature is ↓ minimized for smoother path

There are 3 parts to Optimisation problem:

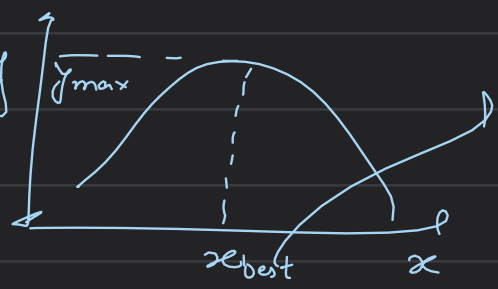
① Objective (or cost) func. ↳ our case: minimizing curve

② Parameters (or Decision Variables) ↳ the var. to be changed to get the best value of objective

③ Constraints: rules or limits which the parameters must satisfy

what is Optimisation? choosing inputs that will result in best possible output ↳ consist of best sol^n to maximise or minimise a problem

eg:



this x_{best} is the optimised sol^n for finding the max y value

⊛ optimisation of input such that you get the best way to get the maximum or minimum output

Objective function: ↳ the value that is to be optimised (in our case curvature of the path)

eg: minimize cost, maximize speed, minimize weight etc

Area = a×b , f(x) = Area

Parameters (Decision Variables): values the optimizer can change to best way optimize f(x), eg: Parameters: a, b

Gradients: slope : Df ↳ by differentiation

Constraints: eg: $a \times b \leq 5$ ↳ a constraint

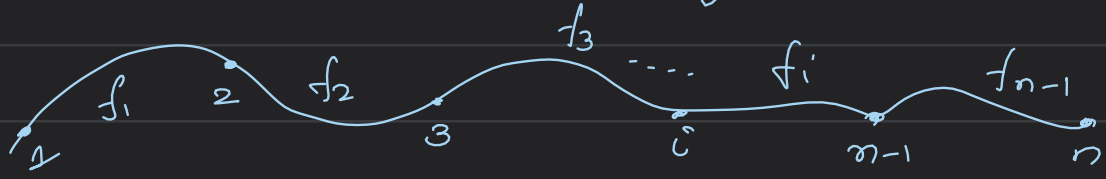
or $a + b = 10$

Method of gradient Descent: we optimize the waypoints to get a new set of waypoints which are then undergone cubic spline interpolation to get a new curve

Step-1: to cubic spline interpolation to get all the polynomials

$f_i(x) = a_i + b_i(x - x_i) + c_i(x - x_i)^2 + d_i(x - x_i)^3$

$i = 1$ to $n-1$ $\quad n-1$: no. of polynomials, $\quad n$: total no. of waypoints



Step-2: put $x(t) = t$, $y(t) = f(t)$

& compute $|K(t)| = \dfrac{|x''(t)y'(t) - y''(t)x'(t)|}{((x'(t))^2 + (y'(t))^2)^{3/2}}$

$|K(t)|$ ↳ curvature

Step-3: compute total curvature: $K_i(t)$ ↳ curvature of polynomial $f_i(x)$

$J = \sum_i (K_i(t))^2$

↳ total curvature

Step-4: now we have to optimise the waypoints to get the minimum total curvature

⇒ slightly move the waypoints to get the minimum curvature

↳ How much ?

$y_i = y_i - \text{eta} * \text{gradient}$

↳ learning rate ⇒ const

$\downarrow\downarrow$

y_{opt}

$\dfrac{\partial J}{\partial y_i} = \dfrac{J(y_i + \varepsilon) - J(y_i)}{\varepsilon}$ ↳ movement

how sensitive the curvature is to waypoint

logic: waypoint with sharp bend → large $\frac{dJ}{dy_i}$

gradient points towards inc curvature

so $J_{new} << J_{old}$ peak flattens

Step-5: we will get a new set of way points $(x_i, y\text{-}opt)$

↓

recompute the spline

Task-1: curvature optimization for 4 points

curvature_optimization > curvature_optimization > gradient_descent.py > main

```python
import matplotlib.pyplot as plt
import numpy as np

eta = 0.01

def main(args=None):
    x = np.array([1, 2, 3, 4])
    y = np.array([1, 3, 2, 3])
    A = np.zeros((12, 12))
    B = np.zeros(12)
    eqn = 0
    for i in range(3):
        A[eqn, 4*i] = 1
        B[eqn] = y[i]
        eqn += 1
        dx = x[i+1] - x[i]
        A[eqn,4*i] = 1
        A[eqn, 4*i+1] = dx
        A[eqn, 4*i+2] = dx**2
        A[eqn, 4*i+3] = dx**3
        B[eqn] = y[i+1]
        eqn += 1
    for i in range(1,3):
        dx = x[i] - x[i-1]
        A[eqn, 4*(i-1)+1] = 1
        A[eqn, 4*(i-1)+2] = 2*dx
        A[eqn, 4*(i-1)+3] = 3*dx**2
        A[eqn,4*i+1] = -1
        eqn += 1
```

→ 3×4 → unknowns

→ a 12×12 matrix

→ 12×1 matrix

$f_i(x_i, y_i) = 0$

↳ the waypoint satisfy the eqn

first Derivative continuity

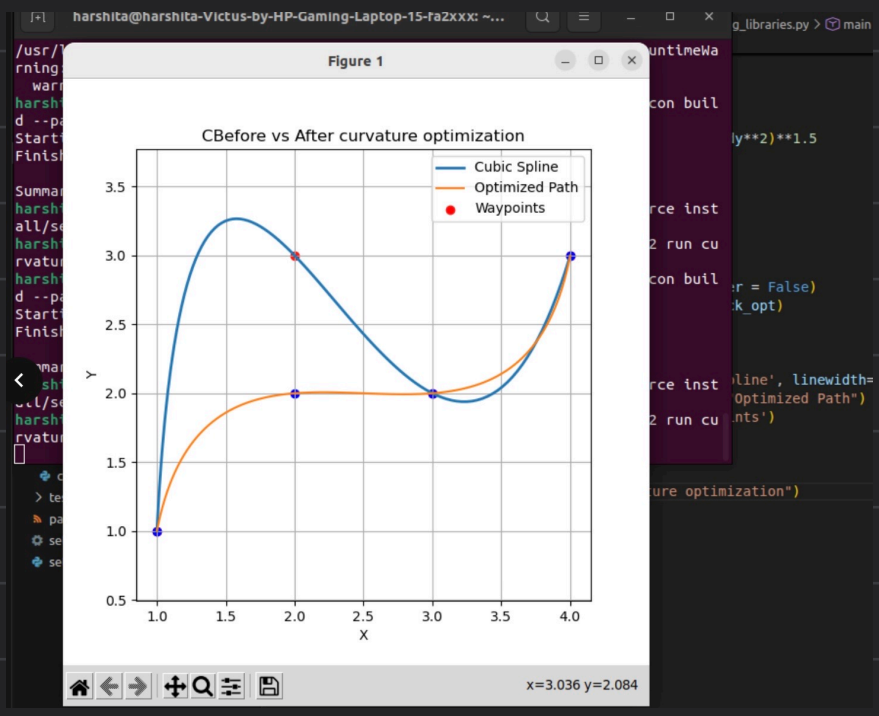
```
    for i in range(1,3):
        dx = x[i] - x[i-1]
        A[eqn, 4*(i-1)+2] = 2
        A[eqn, 4*(i-1)+3] = 6*dx
        A[eqn, 4*i+2] = -2
        eqn += 1
    A[eqn, 2] = 2
    eqn += 1
    dx = x[3] - x[2]
    A[eqn, 10] = 2
    A[eqn, 11] = dx*6
    eqn += 1
    coeffs = np.linalg.solve(A, B)

    def computePoly(i, xi):
        a = coeffs[4*i]
        b = coeffs[4*i+1]
        c = coeffs[4*i+2]
        d = coeffs[4*i+3]
        return a + b*(xi-x[i]) + c*(xi-x[i])**2 + d*(xi-x[i])**3

    X, Y = [], []
    for i in range(3):
        xi_vals = np.linspace(x[i], x[i+1], 50)
        yi_vals = computePoly(i, xi_vals)
        X.extend(xi_vals)
        Y.extend(yi_vals)
```

```
def computeCurvature(y_input):

  B_temp = np.zeros(12)

  eqn = 0

  for i in range(3):

    B_temp[eqn] = y_input[i]
    eqn += 1

    B_temp[eqn] = y_input[i+1]
    eqn += 1

  eqn += 2
  eqn += 2

  B_temp[eqn] = 0
  eqn += 1

  B_temp[eqn] = 0

  coeffs_temp = np.linalg.solve(A, B_temp)

  total_curvature = 0

  for i in range(3):

    xi_vals = np.linspace(x[i], x[i+1], 50)

    for xi in xi_vals:

        b = coeffs_temp[4*i+1]
        c = coeffs_temp[4*i+2]
        d = coeffs_temp[4*i+3]

        yd = b + 2*c*(xi - x[i]) + 3*d*(xi - x[i])**2
        y2d = 2*c + 6*d*(xi - x[i])

        kappa = abs(y2d)/(1 + yd**2)**1.5

        total_curvature += kappa**2

  return total_curvature

J = computeCurvature(y)
```

```
    def computeGradient(index):
      eps = 0.001
      J1 = computeCurvature(y)
      y_temp = y.copy()
      y_temp[index] += eps
      J2 = computeCurvature(y_temp)

      print(J1)
      print(J2)
      print(J2 - J1)

      gradient = (J2 - J1)/eps
      return gradient

    grad1 = computeGradient(1)
    grad2 = computeGradient(2)

    print(grad1)
    print(" ")
    print(grad2)
    print("\n")


    y_opt = y.copy()

    y_opt[1] -= eta*grad1
    y_opt[2] -= eta*grad2

    B_opt = np.zeros(12)

    eqn = 0
    for i in range(3):

      B_opt[eqn] = y_opt[i]
      eqn += 1

      B_opt[eqn] = y_opt[i+1]
      eqn += 1

    coeffs_opt = np.linalg.solve(A, B_opt)

    X_opt = []
    Y_opt = []
```

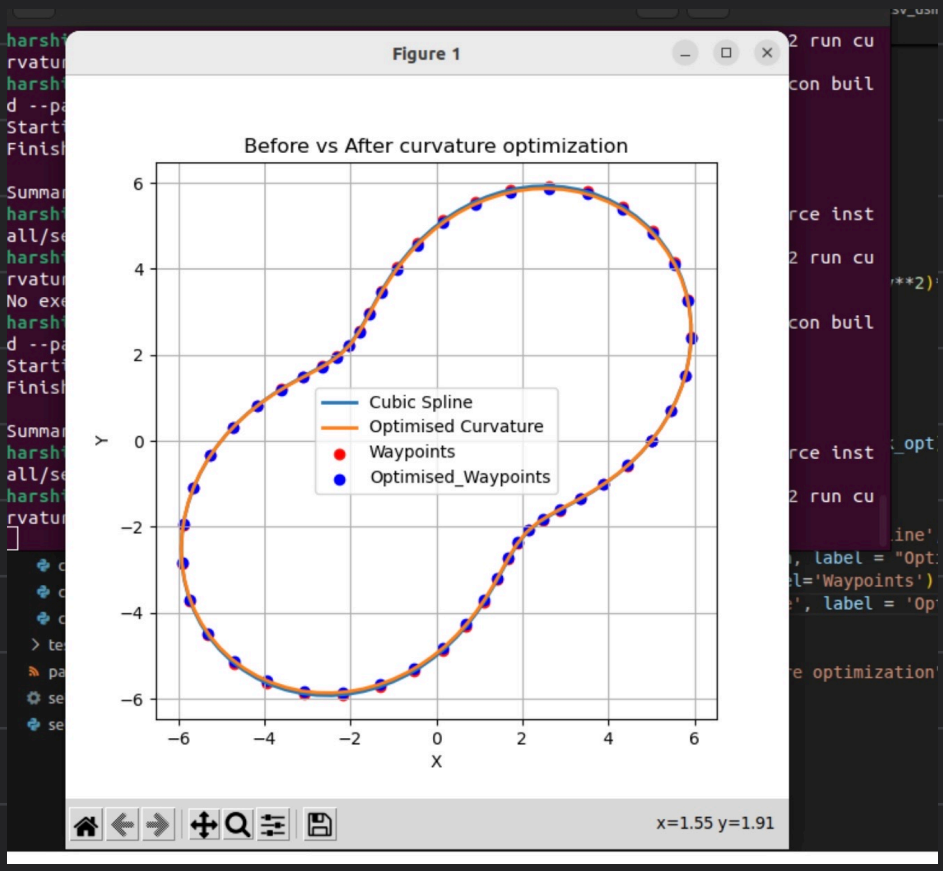
```
    for i in range(3):

      xi_vals = np.linspace(x[i], x[i+1], 50)

      a = coeffs_opt[4*i]
      b = coeffs_opt[4*i+1]
      c = coeffs_opt[4*i+2]
      d = coeffs_opt[4*i+3]

      yi_vals = (
       a
       + b*(xi_vals-x[i])
       + c*(xi_vals-x[i])**2
       + d*(xi_vals-x[i])**3
)
      X_opt.extend(xi_vals)
      Y_opt.extend(yi_vals)

    plt.plot(X, Y, label="Original Spline")
    plt.plot(X_opt, Y_opt, label="Optimized Spline")
    plt.scatter(x, y, color='blue')
    plt.scatter(x, y_opt, color='red')
    plt.legend()
    plt.show()
```

Task-2: Curvature Optimization using libraries




```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.interpolate import splprep, splev

def main(args=None):
    eta = 0.01
    x = np.array([1, 2, 3, 4])
    y = np.array([1, 3, 2, 3])

    tck, u = splprep([x, y], s=0, per=False)
    u_new = np.linspace(0, 1, 100)
    x_smooth, y_smooth = splev(u_new, tck)

    dx, dy = splev(u_new, tck, der=1)

    d2x, d2y = splev(u_new, tck, der=2)

    kappa = np.abs(dx*d2y - dy*d2x) / (dx**2 + dy**2)**1.5
    J = np.sum(kappa**2)

    y_opt = y.copy()

    y_opt[1] -= eta
    y_opt[2] += eta

    tck_opt, u_opt = splprep([x, y_opt], s=0, per = False)
    x_opt_smooth, y_opt_smooth = splev(u_new, tck_opt)

    plt.figure(figsize=(6, 6))
    plt.plot(x_smooth, y_smooth, label='Cubic Spline', linewidth=2)
    plt.plot(x_opt_smooth, y_opt_smooth, label="Optimized Path", linewidth = 2)
    plt.scatter(x, y, color='red', label='Waypoints')
    plt.scatter(x, y_opt, color='blue')
    plt.xlabel("X")
    plt.ylabel("Y")
    plt.title("Before vs After curvature optimization")
    plt.legend()
    plt.grid()
    plt.axis('equal')
    plt.show()
```

Task. 3: Curvature optimisation of csv using libraries




```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.interpolate import splprep, splev
import os

def main(args=None):
    eta = 0.01
    csv_path = os.path.expanduser('/home/harshita/Downloads/loop_track_waypoints.csv')

    dataPoints = pd.read_csv(csv_path)
    x = dataPoints['X'].values
    y = dataPoints['Y'].values

    if (x[0] != x[-1]) or (y[0] != y[-1]):
        x = np.append(x, x[0])
        y = np.append(y, y[0])

    tck, u = splprep([x, y], s=0, per=True)
    u_new = np.linspace(0, 1, 100)
    x_smooth, y_smooth = splev(u_new, tck)

    dx, dy = splev(u_new, tck, der=1)
    d2x, d2y = splev(u_new, tck, der=2)

    kappa = np.abs(dx*d2y - dy*d2x) / (dx**2 + dy**2)**1.5
    J = np.sum(kappa**2)

    y_opt = y.copy()
    y_opt[1:-1] -= y_opt[1:-1] * eta

    tck_opt, u_opt = splprep([x, y_opt], s=0)
    x_opt_smooth, y_opt_smooth = splev(u_new, tck_opt)
```

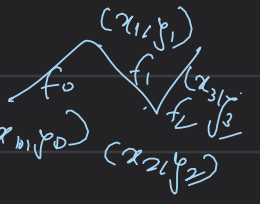
```python
    plt.figure(figsize=(6, 6))
    plt.plot(x_smooth, y_smooth, label='Cubic Spline', linewidth=2)
    plt.plot(x_opt_smooth, y_opt_smooth, label = "Optimised Curvature", linewidth = 2)
    plt.scatter(x, y, color='red', label='Waypoints')
    plt.scatter(x, y_opt, color = 'blue', label = 'Optimised_Waypoints')
    plt.xlabel("X")
    plt.ylabel("Y")
    plt.title("Before vs After curvature optimization")
    plt.legend()
    plt.grid()
    plt.axis('equal')
    plt.show()
```

$$f_i(x) = a + b(x-x_i) + c(x-x_i)^2 + d(x-x_i)^3$$

$$f_0(x) = a + b(x-x_0) + c(x-x_0)^2 + d(x-x_0)^3 \qquad i = 0,1,2$$

$$f_1(x) = a + b(x-x_1) + c(x-x_1)^2 + d(x-x_1)^3$$

$$f_2(x) = a + b(x-x_2) + c(x-x_2)^2 + d(x-x_2)^3$$



$f \rightarrow f_{opt}$ ① Compute curvature

by gradient descent

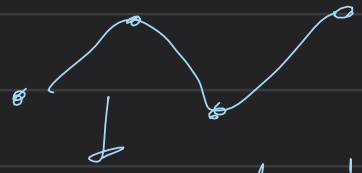
$$f = f - \eta \frac{\partial J}{\partial f} \qquad (\eta)$$

$y_i = f_i(x)$

f_{opt}

$$J = \sum |K|^2$$

$$K_i = \frac{|y_i''(x)|}{(1+(y_i'(x))^2)^{3/2}}$$



we have to change

$f = [y_0, y_1, y_2, y_3]$

f_{opt} such that the curvature is optimized

$$J = \left(\frac{y_0''(x)}{(1+y_0(x)^2)^{3/2}}\right)^2 + \left(\frac{y_1''(x)}{(1+(y_1(x))^2)^{3/2}}\right)^2 + \left(\frac{y_2''(x)}{(1+(y_2(x))^2)^{3/2}}\right)^2$$

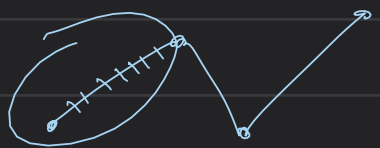
① graph-1 : $(x, y) \rightarrow (x, Y)$

② graph-2 : $(x, y_{opt}) \rightarrow (x, Y_{opt})$

$$J = K_0^2 + K_1^2 + K_2^2$$

$$(y_{opt})_0 = y_0 - \frac{(J(y_0+\eta) - J(y_0))}{\eta}\eta$$

$$(y_{opt})_1 = y_1 - \frac{(J(y_1+\eta) - J(y_1))}{\eta}\eta$$



$$(y_{opt})_2 = y_2 - \frac{(J(y_2+\eta) - J(y_2))}{\eta}\eta$$

K_i curvature at any point (x_i, y_i)

$Y_{opt} = [(y_{opt})_0, (y_{opt})_1, (y_{opt})_2]$

$Y = [y_0, y_1, y_2]$

from cubic spline interpolation

x_1

$$K_1(x) = \left(\frac{y''(x)}{(1+(y'(x))^2)^{3/2}}\right)$$

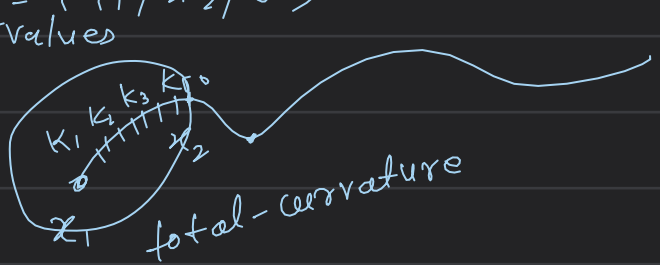
$$y'(x) = b + 2c(x - x_i) + 3d(x - x_i)^2$$

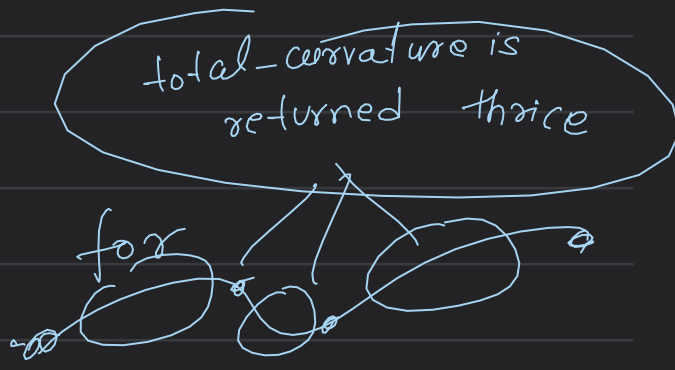
$$y''(x) = 2c + 6d(x - x_i)$$

for waypoint (x_1)

$$K_i(x) = \frac{2c + 6d(x - x_i)}{\left(1 + (b + 2c(x - x_i) + 3d(x - x_i)^2)^2\right)^{3/2}}$$

x_i = values $(x_1, x_2, \ldots)$





J = compute curvature (y)

↓

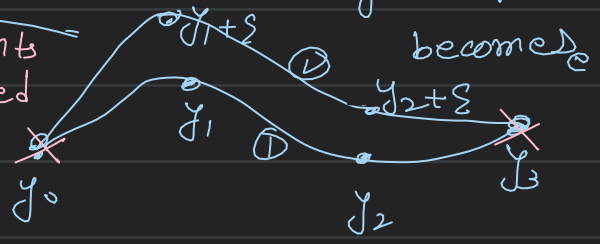
compute total curvature of y_0

$$y \xrightarrow{-opt} y_0 - \eta \frac{\partial J}{\partial y} \rightarrow \frac{J \text{ at } y_0 + \varepsilon \; - \; J \text{ at } y_0}{\varepsilon}$$

↓

new choosen waypoint

So on changing y y_0 to $(y_0 + \varepsilon)$ our original splin becomes



So for computation of $\frac{\Delta J}{\Delta y}$ we are suppose to again find new coeff

so that we can find new $y'(x)$ & $y''(x)$ thus new K_i

thus new $J \rightarrow J'$

$$\frac{\partial J}{\partial y} = \frac{J' - J}{\varepsilon}$$

① is the original spline,

② is just a slightly diff spline created just to compute $\frac{\partial J}{\partial y}$

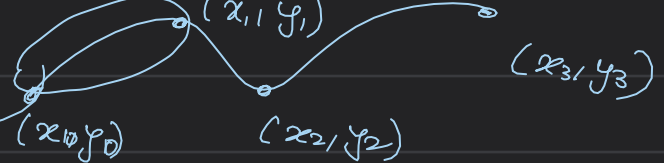
now we will find $y\text{-opt} = y - \eta \frac{\partial J}{\partial y}$

for a particular waypoint

↓

$y\text{-opt}$ → new y-coor of all the waypoints ST the curvature is minimized

② compute gradient:



for say way point (1) we have to consider this part of the curve or f_0 & to find total curvature of all the 50 points for this part (J)

Note: curvature is a property of a point $(K_1 \text{ ---- } K_{50})$

$\propto (x - x_i)$ ↓ x_i : that choosen way point

all 50 points in b/w

```
def computeGradient(index)
    : is basically
    J1 = computeCurvature(y0)
    J'1 = computeCurrature(y0+ε)
    Gradient = (J'1 - J1) / ε
    return Gradient
```

1 → 0, 1, 2 ← no. of gradients to be computed

this fxn returns 3 Gradients

* Gradient are of total curvature ↓ 3 total curvature ↓ 3 Gradients

so computeGradient(0) → Gradient of

computeGradient(1)

computeGradient(2)

Now, compute y-opt ↓

we will compute y-opt for non-end points ⟹ y[1] & y[2] & not of y[0] & y[3]

```
y-opt[0] = y(0)
y-opt[1] = y[1] - eta * computeGradient(1)
y-opt[2] = y[2] - eta * computeGradient[2]
y-opt[3] = y[3]
```

↑ our set of optimized points ← now we will calculate the spline & thus get the optimized curve